Designing a Programming Language for a Small Model: What the Generated Program Should Not Contain

Keywords: domain-specific languages; language design; small language models; program synthesis; runtime contracts; code generation

When a small language model compiles word problems into executable programs, the target language becomes its curriculum: every construct it must emit is something it can learn wrongly. We examine this design problem through SOP Lang, a dataflow language of named computations, using the programs that fine-tuned models of 0.5B to 1.7B parameters actually wrote, and a prompt-only comparison of abstraction levels on the same models before fine-tuning. Validation scaffolding repeated in every target was the site of a characteristic splice failure; a command that owns an algorithm shrank generated code at equal accuracy; splitting compact computations into more wires coincided with far more execution failures; assertions and output conventions were learned as style and carried into types where they did not apply. Asked only to name operations from a catalogue, the 1.5B and 1.7B untuned models matched plain code in familiar wording and produced exact answers, because the language formatted them, but they lost more than plain code when the problems were reworded. We derive design rules for languages whose programmer is a small model: keep contracts, algorithms, and output conventions in the language; keep plans compact; admit only assertions that are invariants of the answer; and raise the abstraction level towards a small catalogue of general operations whose recognition across varied wordings becomes what the model must learn.

1. Introduction

A 1.7-billion-parameter model was fine-tuned twice with the same recipe, on the same problem statements and reference answers. Only the structure of the book-derived training programs differed: compact in the first run, split into more, smaller steps in the second. Both versions were given the same held-out scheduling problem, with five tasks, two of which run in parallel, a safety buffer, and a deadline.

- The first version wrote seven scalar inputs and two lines of arithmetic, computed 46 minutes, and concluded correctly that the 43-minute deadline was missed.
- The second stored the durations in a keyed object, called a `.values()` method that JavaScript objects do not have, and crashed. It did the same on 41 of the 100 problems of that kind.

Every later 1.7B run whose training data kept the split form also failed to execute on 44 to 94 of those 100 problems.

Program-aided language models delegate computation to an interpreter: the model writes a program and the interpreter executes it exactly [1, 2]. When the model is small and fine-tuned for the purpose, the language it writes is not a neutral medium. The model learns the language from examples. Whatever the examples contain it learns as part of the task: boilerplate, data representations, assertions, and output conventions. The language designer therefore decides, often implicitly, what the model must get right.

Several lines of prior work bear on this problem:

- Domain-specific language design has a long literature on when a special language pays off and how to design one for human users [3].
- Design by contract gives a discipline for stating what a component guarantees and what it requires [4].
- Language models can learn to call external operations instead of computing their results [5].
- Supplying a specialized grammar helps large models generate domain-specific languages in context [6].
- The choice of intermediate representation changes what a pretrained model can generalize [7].

What is missing is evidence about how specific design choices in a target language affect the programs a small, fine-tuned model actually writes.

This paper provides such evidence from one fine-tuning programme. We introduce SOP Lang and describe the programme briefly. We then examine five design decisions; for each we state the principle, the evidence in generated programs and outcome counts, and what the evidence does not show. Several decisions had clear effects on the kind of failure and weaker or no effects on the number of correct answers. We report both.

2. SOP Lang

2.1 Programs as wiring diagrams

A SOP Lang program is a set of named computations called wires. A wire is declared by a line beginning with `@`, its name, and a command, and the lines up to the next declaration are its body. Inside a body, `$name` reads the value of another wire. These references define a directed

dependency graph, which the runtime checks for cycles and executes in dependency order; the order in which wires are written does not matter. A caller asks for one or more output wires by name.

The two basic commands are `literal`, whose body is a JSON value, and `jsEval`, whose body is JavaScript executed in an isolated sandbox that receives copies of its dependencies and returns a value. A compiled solution to a word problem usually records the quantities it extracts in a wire called `slots` and computes the answer from them:

```
@slots literal
{"aMinutes": 7, "bMinutes": 11, "cMinutes":
17,
 "dMinutes": 6, "eMinutes": 12,
 "bufferMinutes": 4, "limitMinutes": 43}

@answer jsEval
const join = Math.max($slots.bMinutes,
$slots.cMinutes);
const minutes = $slots.aMinutes + join +
$slots.dMinutes
  + $slots.eMinutes + $slots.bufferMinutes;
return minutes + " minutes: "
  + (minutes <= $slots.limitMinutes ?
"feasible" : "not feasible");
```

This division gives the model two responsibilities: extract the right values into `slots`, and choose and write the right computation in `answer`. The runtime owns execution, isolation, and resource budgets. A failure can therefore be located: a program that does not parse, a graph that is invalid, a body that throws, a value that is wrong, or a value that is right but expressed differently from the reference.

2.2 Declarative commands

Beyond `literal` and `jsEval`, the language admits commands that own a whole algorithm; their bodies name inputs instead of implementing a procedure. Three were added during the programme.

`graphPath` answers a reachability question over an undirected edge list. Its body names the edges, a start node, and a target, and it returns `yes` or `no`.

`aggregate` filters a list by a declared threshold or divisibility predicate and reduces the kept values to a sum, count, minimum, maximum, or average.

`fraction` reduces a ratio to lowest terms.

Each command validates its inputs and its result, so a well-formed call cannot return a value outside its contract. A later tranche added typed containers: stores that accumulate records in stages, for problems whose solution builds and queries a small table.

2.3 The execution contract and the probe helper

Every `jsEval` body runs with a helper function, `probe(condition, message)`, which throws a structured error when its condition is false. Programs use it to assert what must hold of their inputs and results. The runtime reports a failed probe as an execution error that names the message, so every failed assertion is visible in the evaluation record.

3. The programme in brief

Training data came from two sources. Problem types derived from seven books of reasoning exercises each generated statements from a printed template, with a reference program and an independently computed answer. Procedural generators produced further types from elementary operations on lists of numbers. Each program was executed during dataset verification and its inputs were perturbed. Models from the Qwen2.5-Coder family (0.5B and 1.5B) and Qwen3-1.7B were fully fine-tuned on these programs.

A held-out set of problems from types absent from training was scored in four steps: generate one program per problem, execute it, compare the answer with the reference string, and record the outcome class. We re-executed every saved program of the held-out evaluations through the evaluation code, and every recorded outcome was reproduced. The dataset went through thirteen versions. Most were followed by a training run, and several changed more than one aspect of the data. The generalization behaviour of these models is reported in a companion paper; here we use the generated programs as evidence about language design.

Where a section compares two runs, each condition was trained once. We therefore rely on differences visible in individual programs, on repeated observations in later runs, and on explicitly stated limits. We name the model in each section.

4. Five design decisions

4.1 Generic contracts belong in the executor

In the early data versions every `jsEval` body began with the same scaffolding:

- a local definition of the probe function;
- two assertions that the `slots` wire was a non-empty object;
- a wrapper that evaluated the computation inside an asynchronous closure;
- a closing assertion that the result was not empty.

Parameter assertions specific to each program followed. On the held-out problems of the first full 0.5B run, roughly a quarter of all generated lines were scaffolding or parameter assertions.

This scaffolding was where the model most often broke its programs. Of 80 held-out programs that failed to compile, 58 contained a validation line spliced together from two remembered programs:

```
probe(Number.isInteger(slots.durations.every(
  (value) => value > 0), "every item must
have a positive duration");
```

Here a check that every duration is positive is wrapped inside a check that a single value is an integer, which leaves one parenthesis unclosed. None of this code contributed to the answer.

The programme then made the scaffolding part of the executor: the sandbox provides `probe`, and the runtime validates the `slots` object. Targets from then on contained only the computation and the assertions specific to it. The comparison that would show a benefit is not clean. The spliced assertion appears in the first full run only; seven later runs that still carried the scaffolding did not repeat it. Compile failures on the same 225 held-out book problems varied widely both before the change (from 23 to 80 per run) and after it (42 and 83 in the next two 0.5B runs), and the data version that removed the scaffolding also changed the curriculum. Overall validation accuracy stayed where it was (94.7%, against 94.4% to 95.3% before), but that measure is dominated by trained programs and cannot detect small losses.

The evidence therefore shows that the scaffolding was a place where the model could fail, not that removing it reduced failures. The design argument does not depend on more than that. A construct that must appear identically in every program carries no information about the problem and can only be emitted wrongly. Meyer's design by contract places obligations where they can be checked and guaranteed [4]. For a language whose programmer is a small model, that principle has a corollary: obligations that hold for every program belong to the runtime.

4.2 A command should replace an algorithm, not a line

The `graphPath` command was introduced because reachability questions required the model to write a breadth-first search: an adjacency map, a queue, a visited set, and a test for the target. In one held-out procedural type, the task was to keep the values below a threshold, take the largest, and ask whether it is connected to a target node. The 1.5B model trained before the command existed wrote the search inline:

```
const adjacency = new Map();
for (const edge of slots.edges) { / add both
directions / }
const seen = new Set([current]);
const queue = [current];
while (queue.length > 0) {
  const node = queue.shift();
  if (node === slots.target) break;
  for (const next of adjacency.get(node)) {
    if (!seen.has(next)) { seen.add(next);
queue.push(next); }
  }
}
return seen.has(slots.target) ? "yes" : "no";
```

The 1.5B model trained with the command wrote instead:

```
@stage1 graphPath
from: $stage0
to: $slots.target
edges: $slots.edges
```

Both models solved all 40 problems of that type. On this evidence the command did not improve accuracy: the inline search, seen in the same form many times during training, was already reliable. What changed is how much code could contain an error. About fifteen generated lines of queue discipline became three lines naming inputs. A call that parses cannot contain an unterminated loop or a traversal that forgets the reverse edge.

We state this carefully because the programme's own design documentation once overclaimed it. Between these two runs, overall accuracy rose from 53.8% to 62.4% of 705 problems, and that documentation attributed the rise to the new commands. The generated programs do not support the attribution:

- Only the 40 programs of this one type used a new command, and those were already correct.
- The rise came almost entirely from two other procedural types, a percentage discount and a percentage of the third-largest value. Their programs used no new command.
- These two types were solved in the second run and failed in the first, and the data version that introduced the commands changed other targets as well.

The value of a declarative command must be shown on problems where the inline version fails, and our programme did not isolate such a case. We keep the principle as a statement about error surface. Every recurring algorithm the model must transcribe is a place where transcription can go wrong.

4.3 Keep plans compact

A static check of the programs flagged a class of "bloated" plans: few wires with long bodies. In one data version, shipped plans averaged 2.2 wires and 15 lines of JavaScript, and 14% had at most three wires but more than 25 lines. The obvious engineering response was to refactor these into more wires with shorter bodies, each computing one named intermediate value. The next data version did exactly that for the book-derived types. It kept every statement, reference answer, and dataset size unchanged, and verified that every refactored program reproduced its answer. The number of `jsEval` wires rose by 56%, and the bloated class fell from 14% to 3% of plans.

Table 1 compares the two resulting runs on Qwen3-1.7B.

Table 1. Compact against split training targets for the book-derived types. Same base model (Qwen3-1.7B), recipe, statements, and reference answers; one run each. The compact-target run resumed from an interrupted run. Held-out set of 705 problems, 100 of them scheduling problems.

Measure	Compact targets	Split targets
Wires per shipped plan	2.2	2.8
Programs that parsed and formed a valid graph	100%	100%
Programs that executed	91.9%	80.9%
Execution errors, all held-out problems	57	135
Execution errors, scheduling problems	8	59
Correct answers by exact match	460	448

Both models wrote syntactically perfect programs, and the model trained on split targets failed at run time more than twice as often. Two observations limit what this single comparison shows. First, the procedural targets were identical in both data versions, yet procedural execution failures rose from 12 to 31. That rise measures how much a single retraining can move results by itself, or how the change in one set of targets spills over into others. Second, correct answers changed by only 12. Exact matching accepted none of the 100 scheduling problems in either run, so most of the added failures fell on problems scored wrong anyway.

The history of the programme adds context in both directions. Earlier runs had compact programs for this book but had not yet been taught the procedural scheduling shape, and they failed to execute on 58 to 98 of the 100 problems. A later data version was recorded as reverting the book-derived programs to the compact form, but for this book the revert did not take effect: its programs in that version were byte-identical to the split form. Every run after the recorded revert therefore trained on split programs. The three 1.7B runs had 62, 44 and 94 execution failures on the 100 scheduling problems, and a 0.5B run on the same data had 55. Only one run combined compact programs with the taught shape, and it had 8. The split form is associated with high execution failure in every observed run, but compactness alone was not enough, and the one favourable combination was never replicated.

The scheduling example in the introduction shows one way the failures arise. The reference programs for the evaluated problems were identical in both versions, and keyed collections of durations appeared in the training programs of both. Only the model trained on split targets chose that representation here, where five scalars were natural. It then used a method that a keyed collection has in other languages but not in JavaScript, and it lost the plan: its program took the maximum over all durations instead of computing the chain with its one join.

The lesson is not that decomposition is harmful. Named intermediate values make programs easier to inspect, and some commands, such as containers, need them. The lesson is that every additional wire is an interface the model must generate consistently: a name, a representation, and a reference. A split that does not remove an algorithm from the model's responsibility adds interfaces without taking anything away.

4.4 Assertions are part of the curriculum

Assertions in training programs serve verification: they make a program fail loudly when its inputs or result are wrong, and they are how the dataset verifier caught defective examples. The model learns them as it learns everything else, as part of what a program of a given type looks like. The generated programs show both sides of this.

The harmful side appeared on a held-out rate type whose answers are fractional. One problem reads: "A process has a nominal throughput of 12 cases per hour. An overhead equal to 5% of the nominal throughput produces no useful output. The process runs for 110 minutes. How much useful output is produced?" The reference answer is "20.9 useful cases." The 1.7B model trained on the statement-simplification version of the data wrote (abridged):

```
const rate = $slots.nominalThroughput
  * (100 - $slots.overheadPercent) / 100;
probe(Number.isInteger(rate) && rate > 0,
  "the useful rate must be a whole positive
rate");
const hours = $slots.minutes / 60;
probe(Number.isInteger(hours) && hours > 0,
  "the running time must be a whole positive
hours");
return rate * hours + " " + $slots.unit +
".";
```

The computation is right: without the assertions the program returns "20.9 cases." The first assertion stops it, because the model had copied whole-number guards from types in which rates and quantities are counts. For this model, 36 of the 50 problems of the type stopped on an assertion the model had written itself. Run without the assertions, 12 of the 36 print the reference numbers. None would have matched exactly, because the program writes "cases" where the reference says "useful cases". So the borrowed guards stopped correct computations without changing the score, which the output wording had already decided. The model generalized the guard, not its reason.

The useful side appeared on arithmetic compositions that included a newly taught operation. A 0.5B model confused two operations. It read a rank as a percentage in one type and subtracted a discount scaled by 100 in another, and produced wrong values in 64 programs. In every one, the final assertion caught the wrong value and stopped the program before it could be reported: the answer must be a non-negative whole number, which was true of every reference answer of those types. The benefit is diagnostic rather than a gain in accuracy. The failure becomes visible and attributable, but the problem is still unsolved.

The two cases differ in the nature of the assertion. The second states an invariant of the answer; the first states a habit of other types. The programme's resulting policy was to keep only assertions that are invariants of the answer. It deleted parameter checks that restate the generic contract the runtime already enforces, removed guards that are false for some valid input, and turned honest edge cases (nothing qualifies, a tie, an empty filter) into answers rather than errors. Two caveats apply. The policy requires knowing, before training, the answer type of each problem type. And its effect on the rate type was never measured, because a later data version moved that type into training. For language design, the point stands: the assertion vocabulary a language offers, and the assertions its example programs contain, are learned. A language for model programmers should make the right assertion easy to write and the generic ones unnecessary.

4.5 Output conventions are learned across types

The last decision concerns what a program returns. The books from which many problems were derived print their answers as prose, and the first data versions used that prose as the reference answer. For the scheduling type the reference read: "The earliest safe completion time is 46 minutes, so the plan is not feasible. The critical insight is that B and C are parallel branches whose maximum duration controls the join." The compact-target 1.7B model's program computed the right duration and verdict and returned "The earliest safe completion time is 46 minutes, so the plan does not meet the limit." Exact comparison scored it wrong. A retrospective parser accepted the reference sentence template with any of four verdict phrasings. It classified 64 of the 100 outputs of that run as correct, where exact comparison accepted none. We wrote the parser after seeing the outputs, so the count is a diagnostic, not a score.

The explanatory sentence in that reference cannot be derived from the problem. A correct program can reproduce it only by memory, and cannot for a problem it has not seen. The programme therefore rewrote the references of these types into value-bearing forms, such as "46 minutes: not feasible", and moved the prose into a separate explanation file.

That change alone did not make correct programs score as correct. Exact matching on the 100 scheduling problems stayed at zero in every later run. In the statement-simplification run, 54 of the 100 programs computed the right duration and verdict. Yet they printed the sentence of a procedural scheduling type that shared the same solution shape ("The earliest safe completion time is 45 minutes, so the plan does not meet the limit."), not the new compact form. The model had learned its output convention from the nearest type with the same shape, not from the type whose references were changed.

The rule for a model-facing language is therefore stronger than "make outputs derivable from the input". The output convention must be consistent across every type the model can mistake for the one being answered. It should also be defined by the language, for example as a structured result that the runtime formats, rather than by each type's references.

4.6 How abstract should the language be?

The five decisions above all move work out of the generated program. The question they raise is how far to go. At one extreme, the model writes general code; at the other, it only names operations from a catalogue and supplies the values it read, and the runtime produces the code and the formatted answer. A new abstraction level cannot be tested by training models on it here, but its effect on untuned models can be measured by teaching it in the prompt. That isolates what the language asks of a model that has not been trained on any of the three forms.

We gave the three untuned base models (Qwen2.5-Coder 0.5B and 1.5B, and Qwen3-1.7B) the same arithmetic compositions, 3 problems for each of 11 held-out and 4 trained compositions, in three forms.

- **Operator plan.** A catalogue of 23 named operations with their parameters. The model replies with a short plan, such as `{"values": [...], "steps": [{"op": "keepAbove", "threshold": 20}, {"op": "nthLargest", "nth": 3}, {"op": "percentOf", "pct": 50}]}`. The harness executes the plan with the operations' own implementations and formats the answer with the final operation's report. Given the correct plans, the harness reproduces all 520 reference answers of this item set.
- **Plain JavaScript.** One function body that returns the answer.
- **SOP Lang with jsEval bodies.** Taught by a syntax guide.

The operator-plan form was shown three solved trained compositions (one of them a path question); the SOP Lang form three solved examples of trained types (one composition and two others); the JavaScript form one composition. Every problem was posed in the generator's own wording and in a narrative wording like the book problems, with the same instances, numbers,

and references in both. Answers were scored by exact matching and by two blind judges (agreement $\kappa = 0.96$).

Table 2. Held-out compositions solved by untuned models, by the form of program they were asked to write (33 problems per cell, the same in both wordings; exact matches in parentheses).

Model	Form	Generator wording	Narrative wording
0.5B	Operator plan	11 (11)	6 (6)
0.5B	Plain JavaScript	22 (3)	13 (2)
0.5B	SOP Lang, jsEval	5 (0)	10 (0)
1.5B	Operator plan	30 (30)	9 (9)
1.5B	Plain JavaScript	30 (10)	19 (7)
1.5B	SOP Lang, jsEval	13 (3)	1 (0)
1.7B	Operator plan	31 (28)	15 (15)
1.7B	Plain JavaScript	26 (6)	26 (2)
1.7B	SOP Lang, jsEval	23 (3)	6 (0)
Haiku 4.5	Operator plan	33 (33)	33 (33)
Haiku 4.5	Plain JavaScript	33 (6)	33 (6)
Haiku 4.5	SOP Lang, jsEval	31 (28)	33 (6)
Sonnet 5	Operator plan	33 (33)	33 (33)
Sonnet 5	Plain JavaScript	33 (3)	33 (3)
Sonnet 5	SOP Lang, jsEval	33 (3)	33 (3)

The results separate three effects.

- **Code writing.** In the generator's wording, the 1.5B and 1.7B models solved as many problems by naming operations as by writing JavaScript (30 and 31, against 30 and 26), with no code to get wrong; at 0.5B they did not (11 against 22), and a plan could still be invalid.
- **Output conventions.** When the language formats the answer, correct programs produce correct outputs. Almost every operator-plan success was an exact match (30 of 30 and 28 of 31), while most JavaScript successes differed from the reference wording and needed the judges to be recognized (10 of 30 and 6 of 26 exact). This is the rule of Section 4.5 measured directly.
- **Reading.** In the narrative wording the operator plans fell to 9 and 15, below plain JavaScript (19 and 26), which at 1.7B did not fall at all. The model now had to recognize which catalogue operations the prose described, and a small untuned model did that less reliably than it wrote ad hoc code for the same prose. SOP Lang with `jsEval` bodies, taught only in the prompt, was the weakest form at 1.5B and 1.7B in both wordings. It demands both an unfamiliar syntax and code.

Two larger models, Claude Haiku 4.5 and Claude Sonnet 5, received the same problems and instructions in batches through agent sessions, never two wordings of a problem in one session, with their tool use audited. The operator plans and the JavaScript of each wording were answered in the same session, so those two rows are not independent answers. They are shown in the last rows of Table 2 for reference. They solved almost every problem in every form and in both wordings. The narrative wording that defeated the small models did not affect them.

Two design consequences follow:

- **SOP Lang is learnable from a description by a capable model.** Taught only in the prompt, it was as accurate as plain JavaScript for Sonnet and nearly so for Haiku (31 against 33 in generator wording), while for the small models at 1.5B and 1.7B it was the weakest form. What a small model must be trained on, a larger model reads.
- **The format advantage of the higher abstraction level persists at scale.** Operator plans gave exact answers every time, while the larger models' correct JavaScript answers matched the reference wording in only 3 to 6 of 33 problems.

The abstraction level therefore did not change a capable model's accuracy, which was already at ceiling. It changed whether a correct computation produced the output the system expects.

A second sample of 115 held-out problems, which also includes book-derived problems, confirms the last point. Taught SOP Lang in the prompt, the same untuned models solved 4, 19 and 34 of its 55 composition problems and 2, 9 and 1 of its 55 book problems. Fine-tuned students of the same sizes solved 45, 55 and 55 of those composition problems. A language that small models are to write must be learned; it cannot be merely described to them.

For language design, the measurement supports raising the abstraction level of what the model emits, with a condition. Naming simple, general operations and leaving code and formatting to the runtime removed code-writing and output-convention errors, and in familiar wording, at 1.5B and 1.7B, it matched code in accuracy. It moved the difficulty to recognition: mapping unfamiliar wording onto the right operations. A small model's language should therefore consist of a small number of general operations. Its training data should present each operation in many wordings, because recognition, not code generation, becomes the skill to learn. Generated `jsEval` bodies remain useful as an escape hatch for what the catalogue does not cover, not

as the main thing the model writes. A larger model can use the full language from its description. A system that pairs a small model on the catalogue with a larger model for everything else gets both the cheap, exact common case and the robust rare one.

5. Discussion

5.1 The language is the curriculum

The five decisions share one principle. A small model fine-tuned on programs learns their surface as readily as their substance. It reproduces scaffolding, representations, assertions, and output wording in places where they do not apply. A construct that must be emitted correctly but carries no information about the specific problem is pure risk. A construct that does carry information should be expressed in the smallest form that carries it.

Herzig and colleagues found that replacing an executable target with a better-designed intermediate representation improved the compositional generalization of pretrained models [7]. Our observations are consistent with that finding in one respect: the model wrote programs that executed more reliably when the target avoided unnecessary interfaces. Gulwani and colleagues describe the program space as a central dimension of synthesis difficulty [8]. For a learned synthesizer, the language designer shapes that space through what the language can express and through what the example programs habitually contain.

5.2 A checklist for model-facing languages

From the evidence above we propose six checks for a language whose main programmer is a small model.

Check 1. Does any construct appear identically in most programs? Move it into the executor. The evidence shows that such a construct is a site of failure; it does not show fewer failures after its removal.

Check 2. Does the model repeatedly transcribe the same algorithm? Give it a declarative command whose body names inputs. The evidence shows smaller programs at equal accuracy; measure accuracy on problems where the inline version fails.

Check 3. Does a proposed refactoring add wires without removing an algorithm from the model's responsibility? Keep the compact form. The evidence is one paired comparison plus consistent later observations; the compact result was not replicated.

Check 4. Is every assertion in the example programs an invariant of the answer? Delete the rest. The evidence shows borrowed guards stopping correct computations and invariant guards catching wrong ones. The policy itself was not measured.

Check 5. Is the output convention defined once, independent of each type's reference wording? The evidence shows that rewriting references alone did not change what the model printed, while answers formatted by the language were exact.

Check 6. Could the model name an operation instead of writing its code? The evidence shows equal accuracy and exact outputs in familiar wording, at the price of harder recognition in unfamiliar wording. Budget the training data for that recognition.

5.3 Limitations

Each paired comparison rests on one training run per condition. Where later runs bear on a comparison we report them, and we do not attribute small aggregate differences to design choices. Data versions sometimes changed more than one thing, and we have said so where it matters. We did not compare SOP Lang with the obvious alternative target, plain JavaScript or Python emitted by the same model under the same budget. The rules therefore concern design choices within a program-writing target, not whether such a target is better than another. The problems are word problems over small numeric and textual inputs, and the rules may need revision for richer domains. The models are small, and larger models may be less sensitive to surface regularities.

6. Conclusion

Designing a language for a small model means deciding what the model must get right. In the programme studied here, the constructs that caused failures were rarely the computations the problems required. They were the scaffolding every program repeated, the interfaces added by an innocent-looking refactoring, the assertions borrowed from other types, and the output wording learned from the nearest related type. Taking something out of the generated program reliably changed the kind of failure: moving it into the runtime, into a command, into a compact plan, or into a single output convention. It changed the number of correct answers far less often. Taken to its limit, where the model only names operations, the same move removes code and format errors and leaves one hard problem: recognizing which operation a sentence describes. A language designer should expect both effects and measure both. The part of the program that depends on the problem should be the model's only job, and the designer's task is to make that part as small and as learnable as possible.

Availability

The SOP Lang runtime, its command documentation, the training data generators, per-item evaluation records including every generated program quoted here, and the scripts that recompute the reported counts are provided as supplementary material. Coding agents assisted in implementing the software and in

drafting this manuscript; the authors verified every reported figure against the item records.

References

[1] Gao, L., Madaan, A., Zhou, S., Alon, U., Liu, P., Yang, Y., Callan, J., & Neubig, G. (2023). PAL: Program-aided Language Models. *Proceedings of Machine Learning Research*, 202, 10764-10799. https://proceedings.mlr.press/v202/gao23f.html

[2] Chen, W., Ma, X., Wang, X., & Cohen, W. W. (2023). Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=YfZ4ZPt8zd

[3] Mernik, M., Heering, J., & Sloane, A. M. (2005). When and how to develop domain-specific languages. *ACM Computing Surveys*, 37(4), 316-344. https://doi.org/10.1145/1118890.1118892

[4] Meyer, B. (1992). Applying "design by contract". *Computer*, 25(10), 40-51. https://doi.org/10.1109/2.161279

[5] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems*, 36, 68539-68551. https://doi.org/10.52202/075280-2997

[6] Wang, B., Wang, Z., Wang, X., Cao, Y., Saurous, R. A., & Kim, Y. (2023). Grammar Prompting for Domain-Specific Language Generation with Large Language Models. *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 65030-65055. https://doi.org/10.52202/075280-2837

[7] Herzig, J., Shaw, P., Chang, M.-W., Guu, K., Pasupat, P., & Zhang, Y. (2021). *Unlocking Compositional Generalization in Pre-trained Models Using Intermediate Representations* (arXiv:2104.07478). arXiv. https://arxiv.org/abs/2104.07478

[8] Gulwani, S., Polozov, O., & Singh, R. (2017). Program Synthesis. *Foundations and Trends® in Programming Languages*, 4(1-2), 1-119. https://doi.org/10.1561/2500000010